EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Frameworks, tools, tracing, and testing

Instructor:
Arash Saifhashemi

Attendance

- You should Participate **in-person**
  - Will be used for curving
  - If you cannot come in person, try to participate **synchronously on Zoom** with my **approval**
- You should attend **discussion sessions**!

Holidays and Special Events

- We will observe USC holidays that fall on **Fridays**.
- Special Events

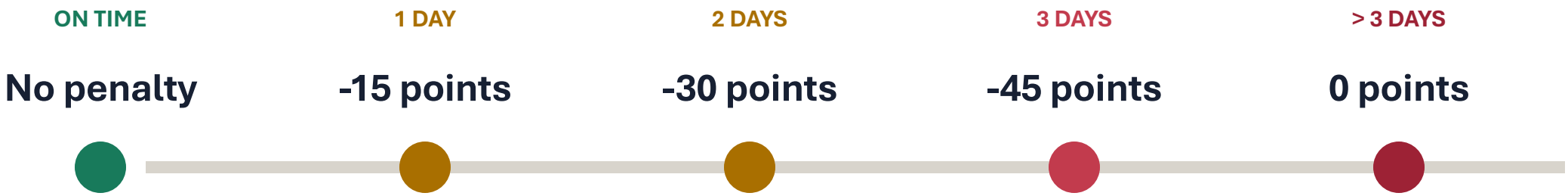
  When there are special events on campus, due to high traffic, **we will have online lectures instead of in-person**. You are required to participate synchronously.

Guest Lectures

- Occasionally we might have guest lectures

  Please make sure you attend either in person or online

Late penalties accumulate by full day.



EMERGENCY EXTENSIONS

Handled case by case. Documentation may be requested, depending on the extension.

Each course platform has one primary job.

Piazza	**Learn and ask**	Lecture material, assignments, questions, and office-hour information
GitHub Classroom	**Submit work**	Push the assignment repository before the deadline
Brightspace	**Read grades**	Official course grades and feedback summaries

Route questions by audience and sensitivity.

Who should see the question?



WHOLE CLASS

Public Piazza post

Generalize the question so everyone can learn from it.

INSTRUCTORS ONLY

Private Piazza post

Use when code context exceeds five public lines.

ONE PERSON

Email

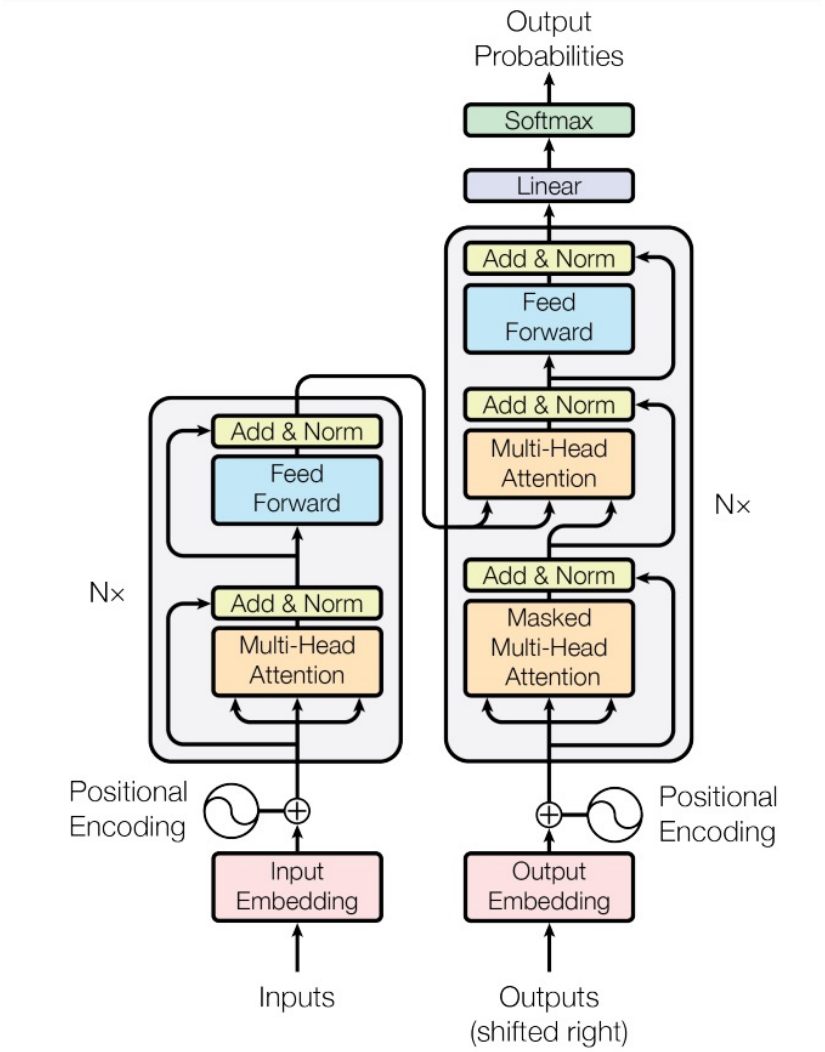
Use for a personal issue or a regrade request to the grader.

PUBLIC POSTS: MAXIMUM 5 LINES OF CODE

Academic Integrity

- All assignments should be your **individual work**
- If you are asking classmates questions beyond the scope of what you could reasonably ask on Piazza in a “public” post, then you are not doing individual work
- **Do not** share your code files or part of your code files with your classmates (current or future students)
- **Do not** post your code on a publicly-accessible website (GitHub, course hero, etc).
- **Do not** step through anyone else’s code, if you need help debugging ask an instructor or TA or Piazza
- If you’re not sure if something is allowed: ask an instructor. Instructors/TAs are always happy to help!

LLMs and Reasoning models



Models generate one token at a time.

TOKEN

A token is a piece of text: a word, part of a word, or punctuation.



Each chosen token becomes context for the next choice.

Generation stops on a signal or a host limit.

TWO STOP MECHANISMS

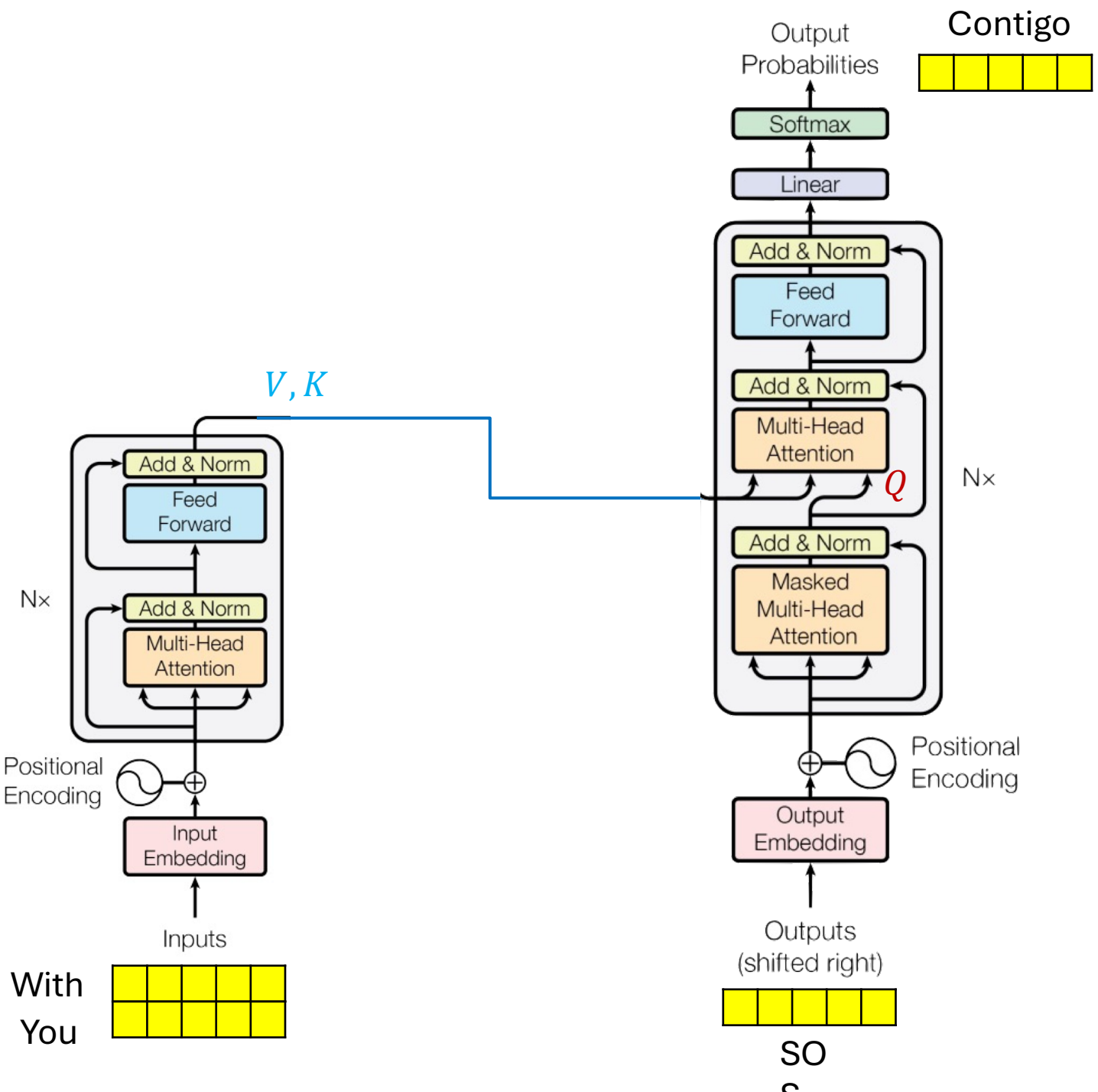


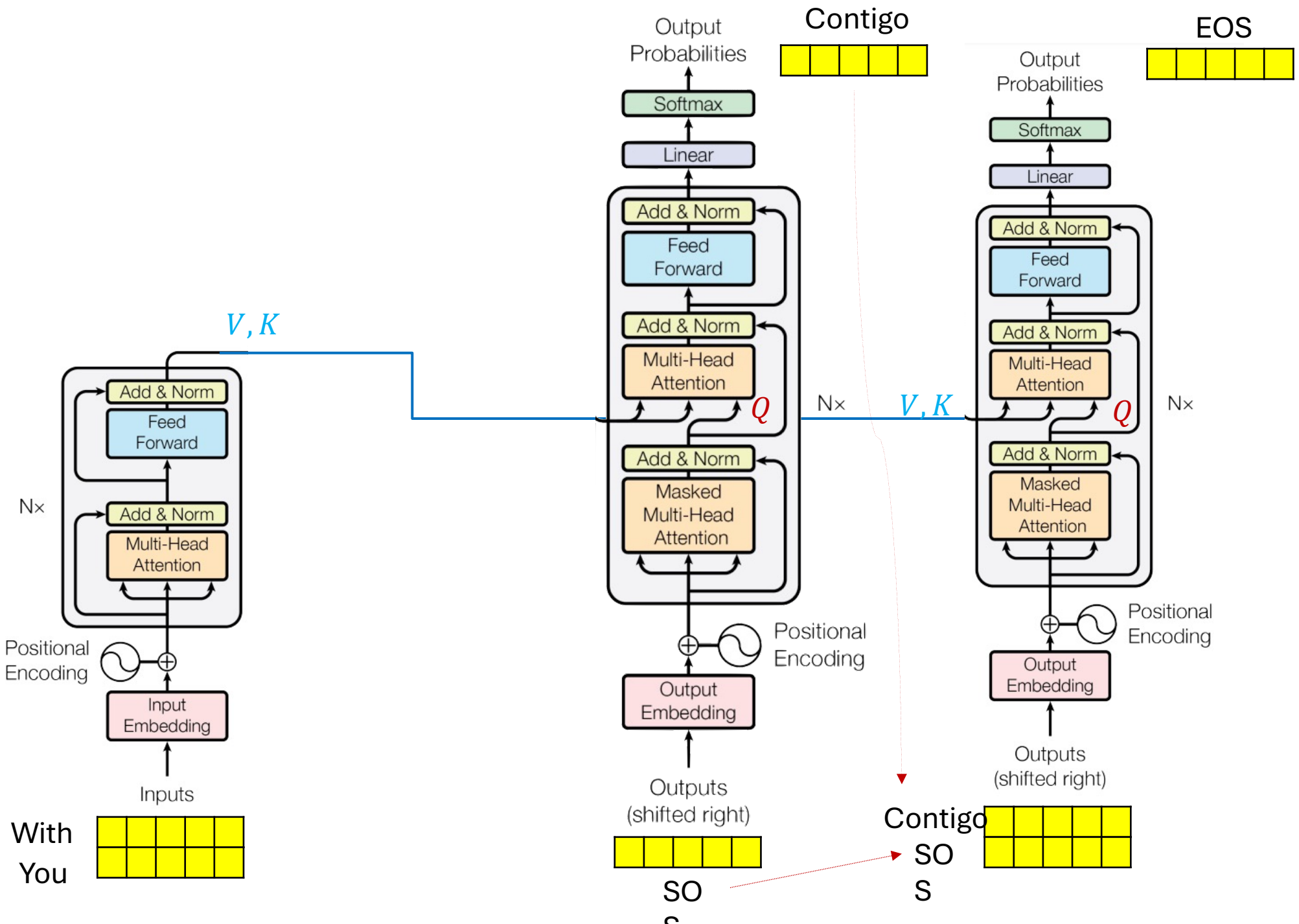
MODEL STOP SIGNAL

An end-of-sequence token marks a natural completion.

HOST LIMIT

The application can also stop at a token, time, or cost limit.


Output Probabilities
Contigo
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Q
Add & Norm
Masked Multi-Head Attention
Nx
Positional Encoding
Output Embedding
Outputs (shifted right)
SO
V, K
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Input Embedding
Inputs
With You


Output Probabilities
Contigo
EOS
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Positional Encoding
Output Embedding
Outputs (shifted right)
V, K
Q
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Positional Encoding
Output Embedding
Outputs (shifted right)
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Inputs
Nx
With You
SOS
Contigo

LLM & AGENT FOUNDATIONS

Milestones in LLM and Agent Development

Selected milestones in architecture, scaling, prompting, multimodality, tool use, and autonomy.

ARCHITECTURE

Transformer Introduced

"Attention Is All You Need" introduces self-attention and parallel training, the technical foundation for modern large language models.

2017

PRETRAINING

BERT & GPT-2 Scale Up

BERT advances bidirectional understanding; GPT-1 and GPT-2 show that generative pretraining and scale improve zero-shot performance.

2018–19

PROMPTING

In-Context Learning

GPT-3 reaches 175B parameters and popularizes few-shot prompting: models learn a task from examples placed directly in the prompt.

2020

ADOPTION

ChatGPT Goes Mainstream

Instruction tuning and RLHF make capable models easier to use. Conversational AI becomes a mass-market product.

2022

TIME CONTINUES

2023

ECOSYSTEM

Open + Multimodal

LLaMA accelerates the open-model ecosystem, while GPT-4 and Gemini push text-and-image reasoning into mainstream systems.

2024

GROUNDING + ACTION

RAG & Tool Use

Retrieval grounds answers in external knowledge. Tool calling lets models search, calculate, and act through APIs.

2025–26

AUTONOMY

Agentic Systems

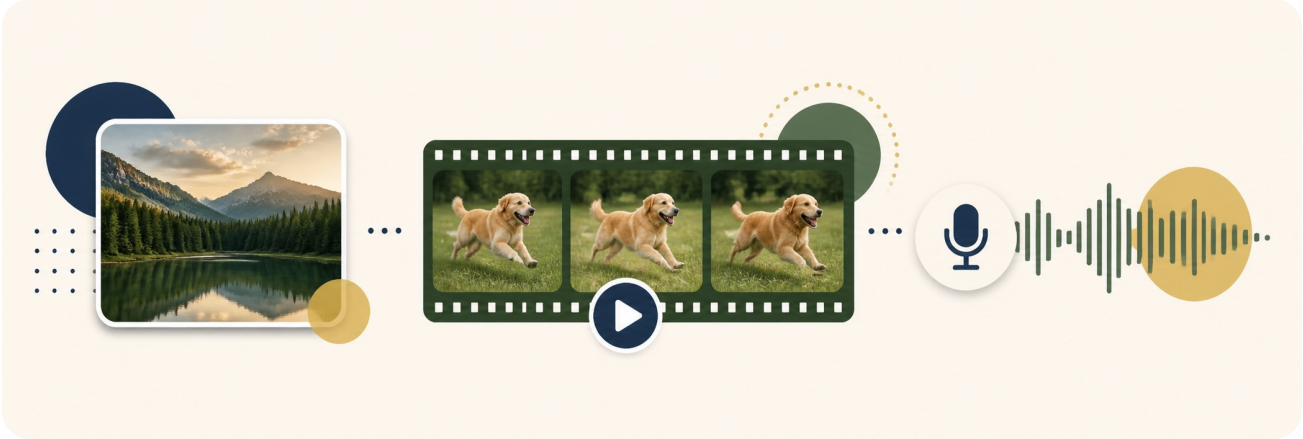
Reasoning models and coding agents plan, use tools, and coordinate multi-step work with increasing autonomy.

Choose models along three dimensions.

1 INFERENCE EFFORT

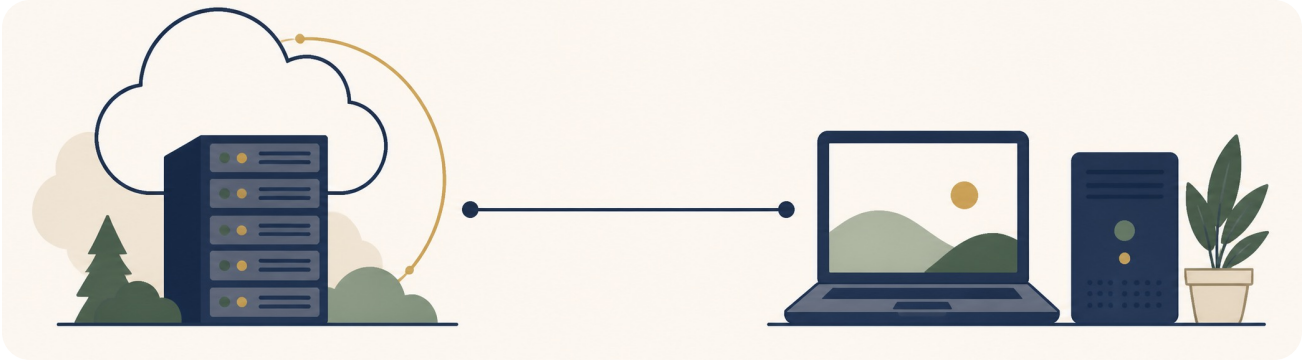
How much computation is spent during this call?

More effort can help multi-step tasks, but usually adds latency and tokens.



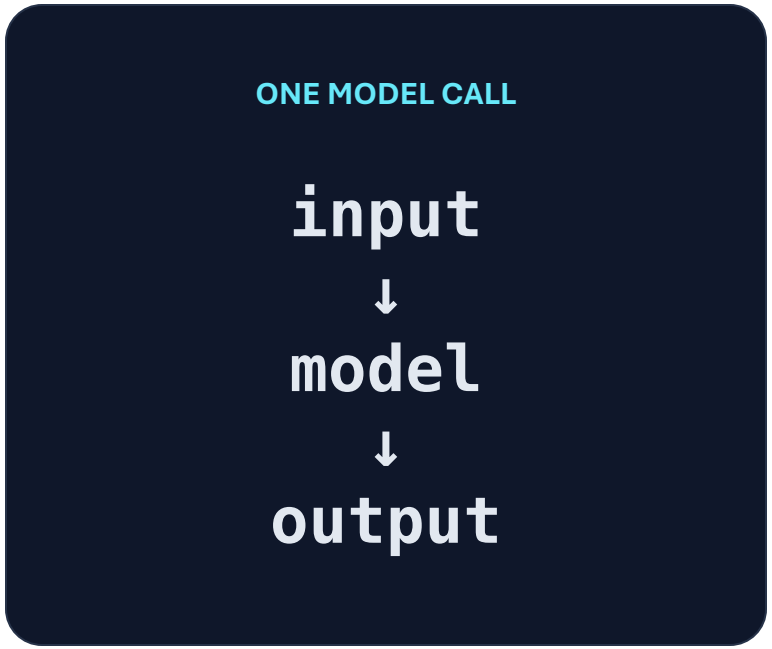
2 MODALITIES

Which input types can the model read: text, image, audio, or video?



3 DEPLOYMENT: HOSTED API OR YOUR HARDWARE

A model call returns output, not an action.



TEXT OR CODE

Generated content

STRUCTURED DATA

Named fields such as JSON

TOOL REQUEST

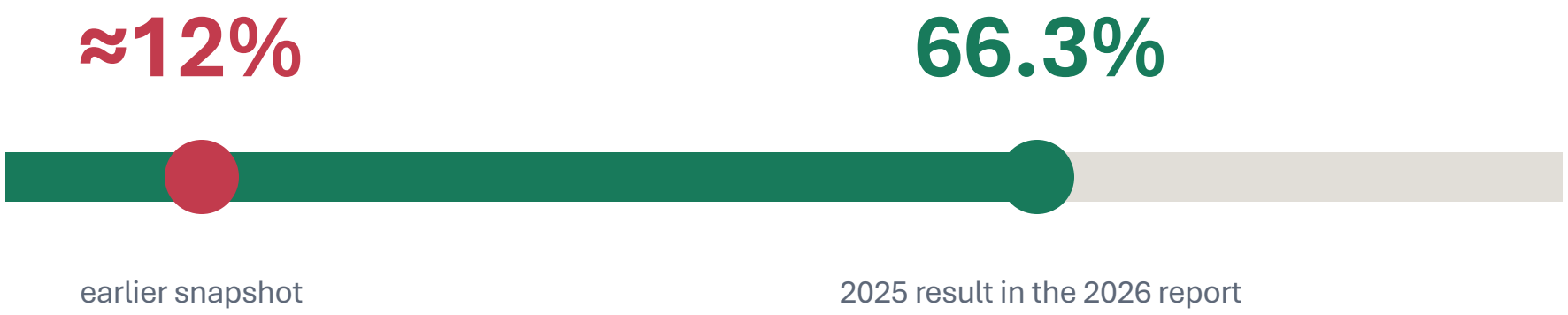
A proposal for your application

A REQUEST IS DATA UNTIL THE HOST ACTS

Strong benchmarks still leave failures.

OSWORLD COMPUTER-USE BENCHMARK

Dated performance snapshot



≈1 in 3

tasks still failed

Rapid gains still require limits, monitoring, and clear stop rules.

A model can answer. An agent can act and use the result.

ASK Compare one response with a task that requires an action.

COMPARE

ANSWER ONLY

“Draft a test plan.”

request → model → response

The task ends after one response.

ACT AND OBSERVE

```
Goal: find one failing test; do not edit.
1  Model: asks to run the tests.
2  App: runs the tests.
3  Result: the failure goes back to the model.
4  Model: explains the failure; the app stops.
```

The test result gives the model new information.

Example: run tests → read the failure → explain it → stop before editing.

An agent acts, observes, and continues.

ANSWER ONLY

“Draft a test plan.”

```
request → model → response → done
```

ACT AND OBSERVE

Goal: find one failing test; do not edit.



Observation means a tool result becomes input to a later model call.

Model, reasoning, and agent are different.

MODEL

LLM

One call maps the current input to a response. It does not act on external systems by itself.

MODEL

Reasoning model

Still an LLM, but designed to spend additional inference-time computation on harder tasks.

APPLICATION

Agent

Software around a model that can request tools, observe results, maintain state, and stop.

The model sees only what your app sends.

LOCAL

files and repository

CURRENT MODEL CALL

CONTEXT

Instructions, conversation messages, included files or images, tool descriptions, and returned tool results

+ learned patterns stored in model parameters

PRIVATE

accounts and credentials

CURRENT

APIs and databases

If evidence is outside the call, the host must fetch it and send it.

Next-token selection repeats until stop.

1. **Read the prefix**
   The current input and generated tokens define the immediate context.

2. **Score candidates**
   The displayed numbers are illustrative, not measured output.

3. **Append and repeat**
   The selected token becomes part of the next prefix.

illustrative_next_token.txt

```
prefix: "Assignment 1 is due at the start of ..."

candidate        example score
Week             0.54
class            0.23
the              0.11

choose "Week"  →  append  →  score again
```

The syllabus supplies the due date.

When is Assignment 1 due?

CHECK 1 / WEEK 2

“In-class work on Assignment 1”

Mentions the work.
Does not give a date.



CHECK 2 / WEEK 3 / DUE-DATE EVIDENCE

“Assignment 1 due at the start of the week.”

EXTRACTED DATE / START OF WEEK 3

Answer: Assignment 1 is due at the start of Week 3.

A visible trace can still contain a wrong step.

SAME TASK: ADD 4 + 5 + 3

MODEL TRACE

$$4 + 5 + 3 = 11$$

VISIBLE BUT WRONG

INDEPENDENT CHECK

$$4 + 5 + 3 = 12$$

Visible trace, wrong sum.
Recalculate.

Check the result with a calculation or test.

Reasoning adds deliberate work.

illustrative_working_summary.txt

```
01 task: "Can these tasks fit in 10 minutes?"
02
03 inspect_files = 4 minutes
04 run_tests     = 5 minutes
05 summarize     = 3 minutes
06
07 total = 4 + 5 + 3  # 12
08 fits  = total <= 10  # False
09
10 answer = "No. The tasks require 12 minutes."
```

1 **Calculate**

Combine the supplied facts instead of guessing from the wording.

2 **Compare**

Use the ten-minute constraint as the decision boundary.

3 **Conclude**

Return a concise answer after the extra inference work.

Reasoning needs evidence.

When is Assignment 2 due?

AVAILABLE EVIDENCE

Question only
No syllabus passage
No tool result

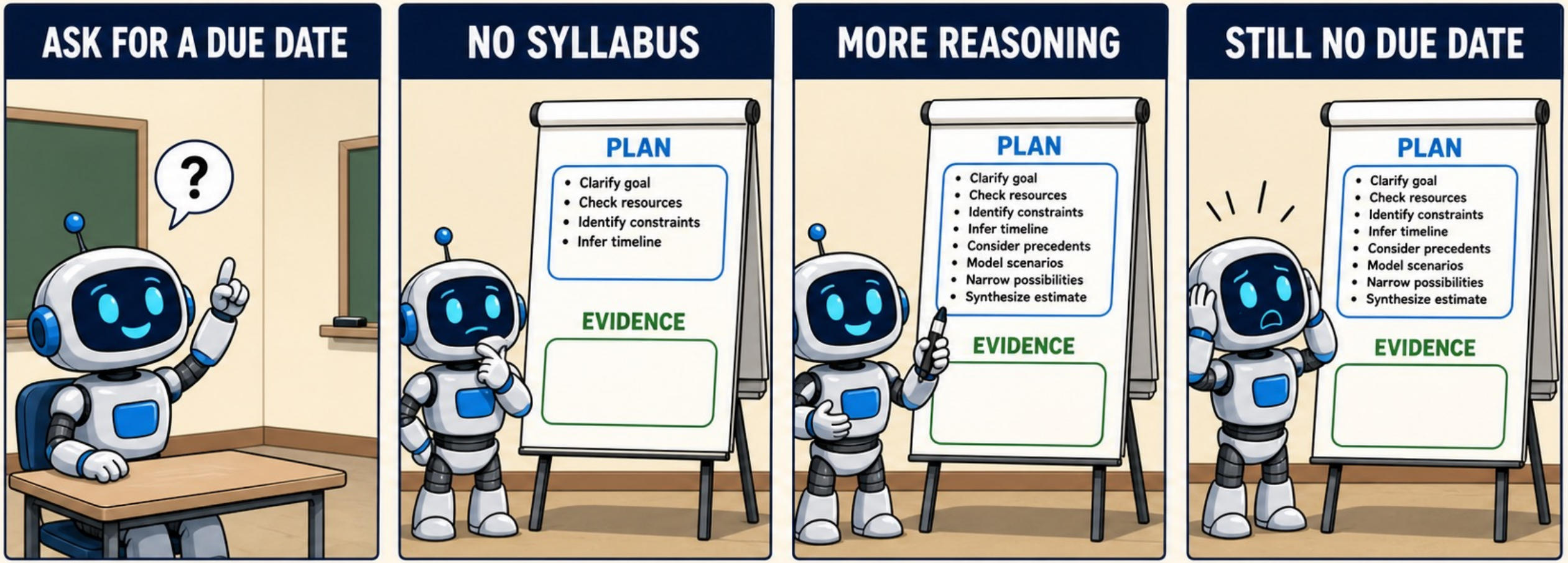
→

CANNOT DETERMINE

Retrieve a source through the host, or state that the fact is unavailable.

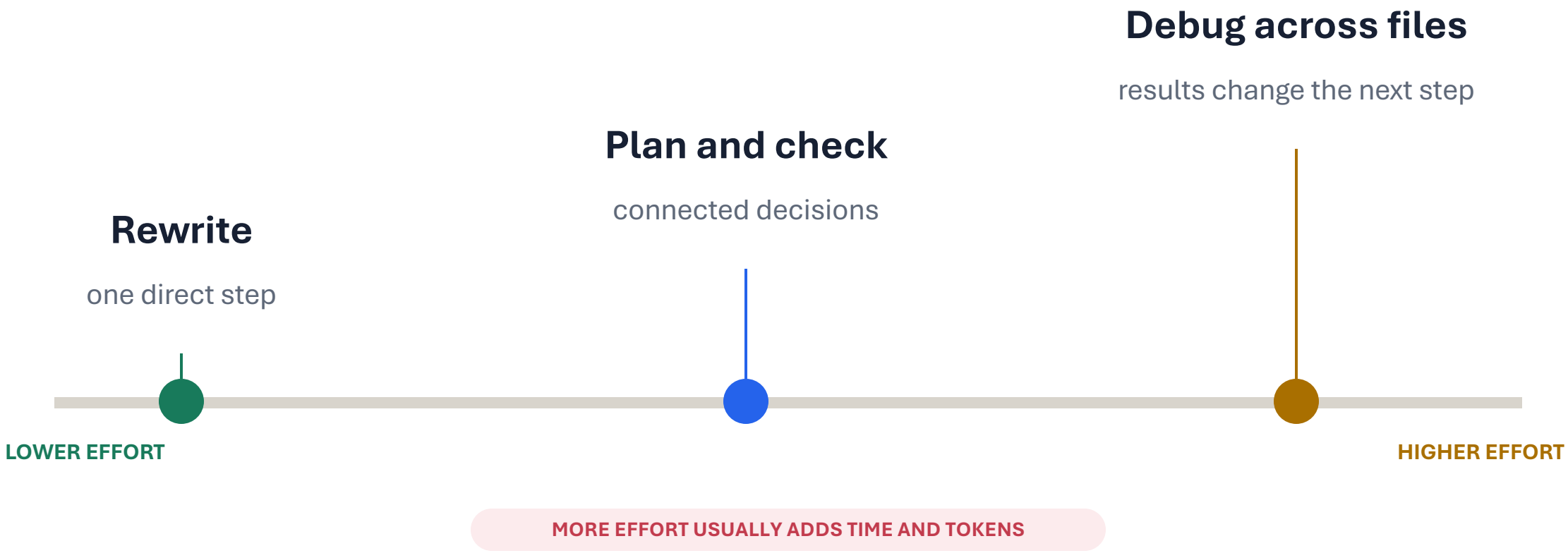
Reasoning uses present evidence; retrieval supplies absent evidence.

REASON HARDER?



Reasoning cannot manufacture evidence.

Use more reasoning for dependent decisions.



An agent is software around a model.



ONE MODEL CALL

```
ask → model → answer → done
```

AGENT LOOP

```
ask → decide → act → observe →
continue or stop
```

A reasoning model is a model choice. An agent is a program design.

Agent = system. Agentic = behavior.

AI AGENT

Course assistant

A concrete system with a model, tools, state, and host code.



AGENTIC BEHAVIOR

Adapt the next step

```
search
↓ evidence missing
search again
↓ evidence sufficient
answer
```

A tool request is a proposal to host code.

tool_contract_and_request.yaml

```yaml
tool:
  name: lookup_assignment
  purpose: read due point and requirements
  input: assignment: integer
  returns: due, requirements, source

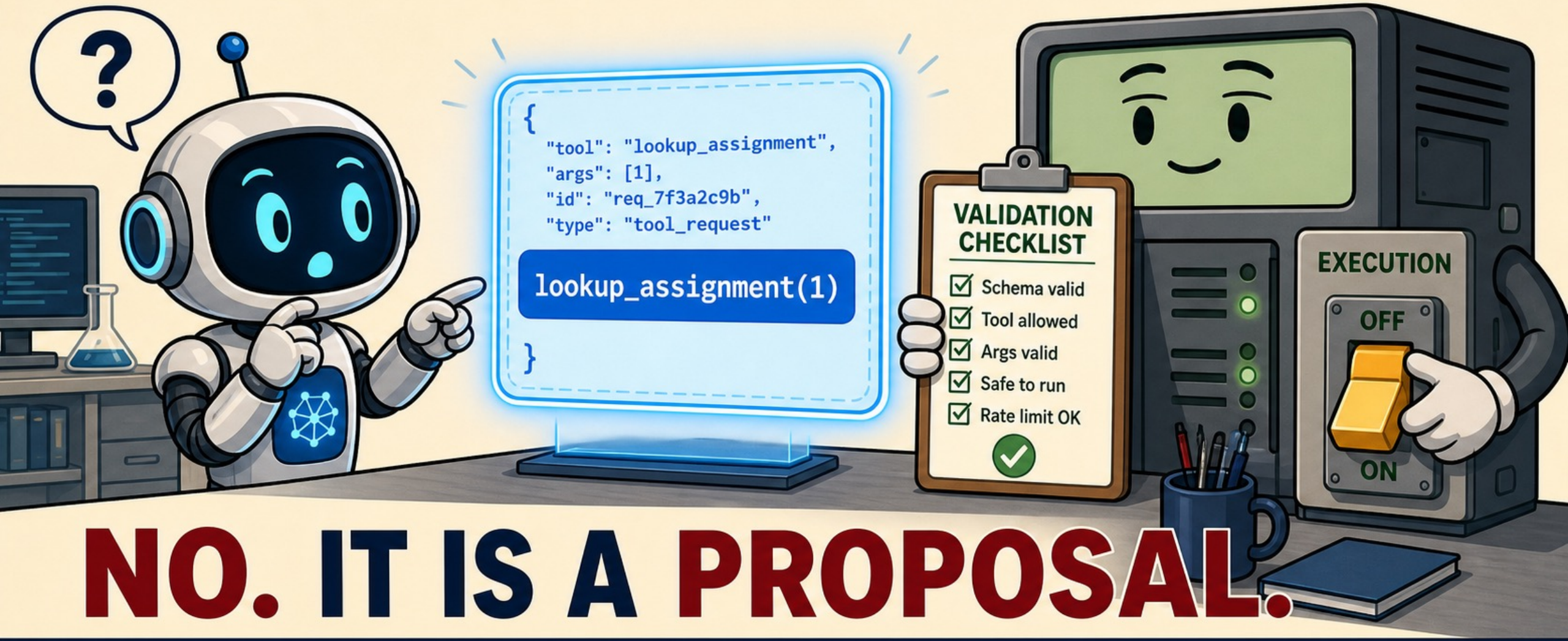
model_output:
  {"name": "lookup_assignment",
   "arguments": {"assignment": 1}}
```

1 **Named capability**
The model sees what the tool does and how to request it.

2 **Grounded result**
The declared result includes a source for downstream evidence.

3 **Candidate arguments**
The JSON is proposed input. It has not executed anything.

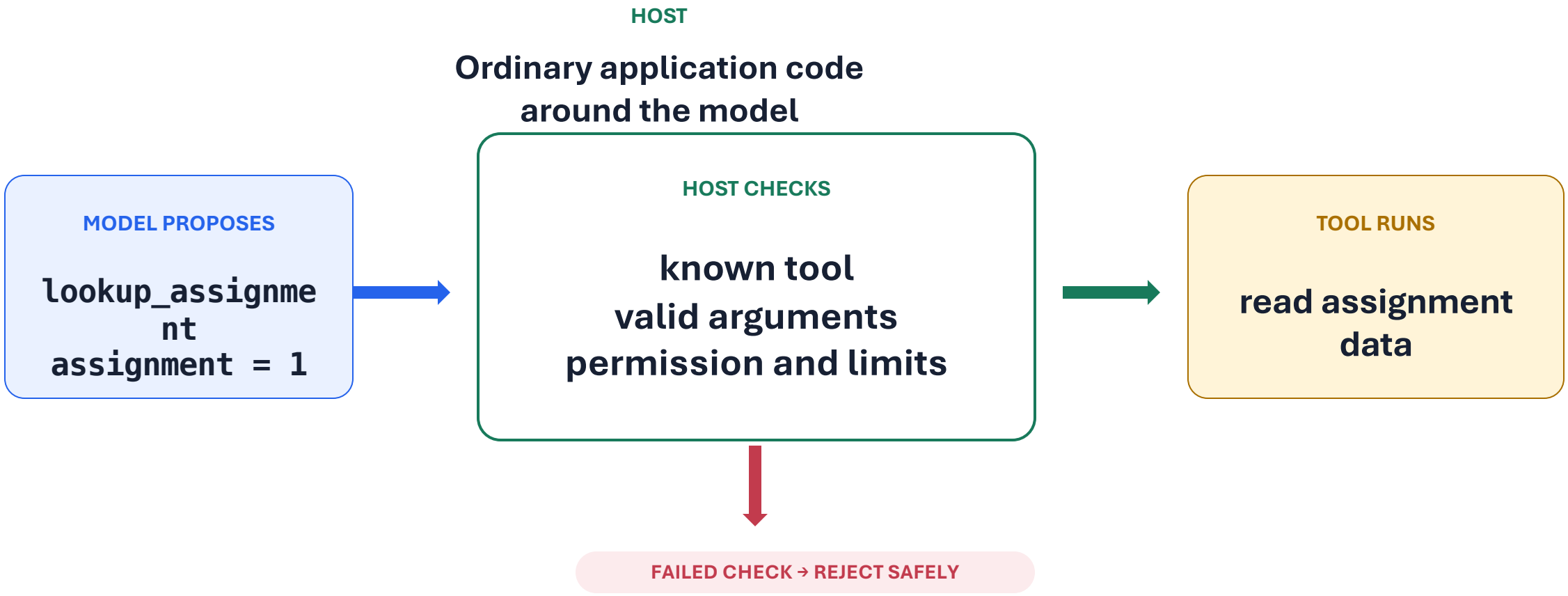
IS THIS AN ACTION?



NO. IT IS A PROPOSAL.

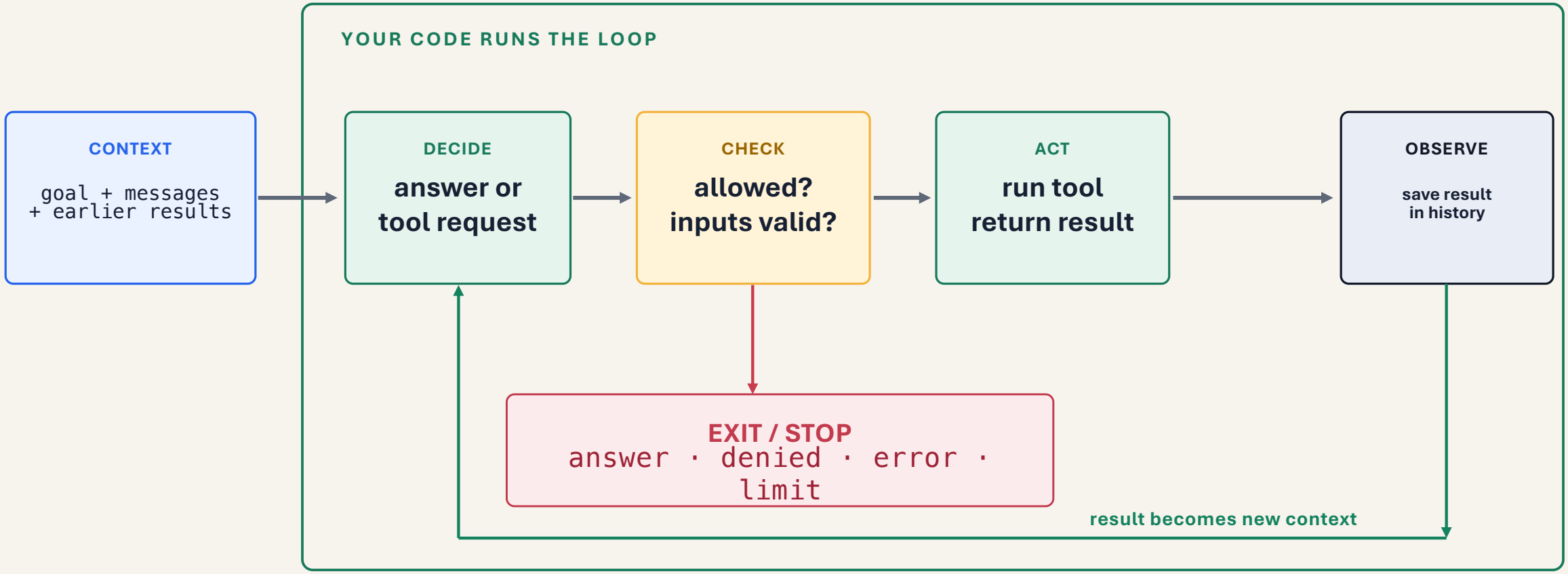
The host validates and executes.

The host decides whether tools run.



The model proposes. The host owns permission, execution, state, and limits.

An agent repeats a simple loop: decide, act, observe.



The host repeats the loop or exits.

The loop is explicit host code.

1 Normal exit
A final answer returns immediately.

3 Tool action
Only an allowed, checked call reaches execution.

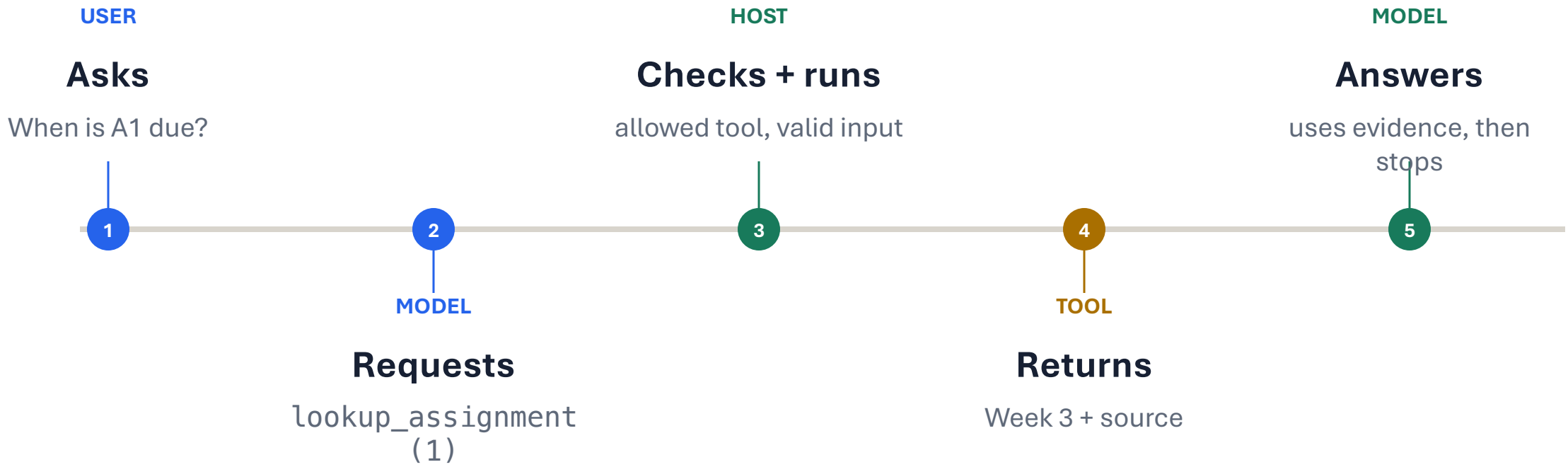
agent_loop.py

```
01 last_call = None
02 for step in range(MAX_STEPS):
03     reply = call_model(messages, tools)
04     if reply.is_final: return reply.text
05     call = check_tool_request(reply)
06     key = (call.name, call.arguments_json)
07     if key == last_call: raise RepeatedToolCall(key)
08     last_call = key
09     result = run_allowed_tool(call)
10     messages += [reply, tool_message(result)]
11 raise StepLimitExceeded(MAX_STEPS)
```

2 Repeat guard
Host code stops identical calls from cycling.

4 Hard limit
The model cannot remove MAX_STEPS.

One agent run turns a tool result into evidence.



The tool result is new context for the second model call.

NO STOP RULES



THE PROMPT SAYS KEEP TRYING.

Stopping is host policy.

Agents need stop rules.

NORMAL COMPLETION

Enough evidence → Return the answer and end the run.

RECOVERABLE FAILURE

Error, timeout, or denied request → Retry within policy, ask the user, or stop clearly.

HARD LIMIT

Max steps, repeat, time, or cost → Stop even if the model asks to continue.

Stopping is host policy, not a suggestion in the prompt.